

API REST para Gestión de Usuarios

Descripción

Esta API REST básica permite la gestión de usuarios mediante operaciones CRUD (Create, Read, Update, Delete), aplicando principios de arquitectura cliente-servidor y buenas prácticas de desarrollo backend. La implementación utiliza **Node.js con Express**, con almacenamiento en memoria para simplicidad (fácil de extender a una base de datos como MongoDB).

Características clave:

- Validación de datos de entrada.
- Manejo de errores HTTP (e.g., 400 Bad Request, 404 Not Found).
- Encriptación de contraseñas con **bcrypt**.
- Identificadores únicos con **UUID**.
- Estructura modular: separación de modelos, controladores y rutas.
- Soporte para CORS, permitiendo integración con clientes como aplicaciones móviles.

Versión de la API: v1

Base URL: `http://localhost:3000/api`

Puerto por defecto: 3000

Esta API es ideal para talleres educativos o prototipos, demostrando control de versiones (recomendado con Git) y pruebas con herramientas como Postman.

Estructura del Proyecto

El proyecto sigue una arquitectura MVC (Model-View-Controller) adaptada para APIs REST, con separación clara de responsabilidades:

```
proyecto/
├── app.js                # Archivo principal: configuración del servidor
Express y middleware.
├── package.json         # Dependencias y scripts de npm.
├── models/
│   └── usuario.js       # Modelo: lógica de datos (CRUD en memoria,
validaciones básicas).
├── controllers/
│   └── usuarioController.js # Controladores: manejo de requests/responses y
lógica de negocio.
├── routes/
│   └── usuarioRoutes.js  # Rutas: definición de endpoints HTTP.
└── README.md            # Esta documentación.
```

- **Modelos:** Manejan el almacenamiento y operaciones de datos (simulación de DB en array).
- **Controladores:** Procesan la lógica, validan schemas y llaman al modelo.
- **Rutas:** Expone los endpoints y enlaza con controladores.

Documentación Técnica

Entidad: Usuario

La entidad **Usuario** representa un registro con los siguientes campos:

Campo	Tipo	Descripción	Requerido	Notas
id	UUID	Identificador único.	Sí (auto-generado)	Generado con uuid .
nombre	String	Nombre completo del usuario.	Sí	Mínimo 2 caracteres, máximo 50.
email	String	Correo electrónico (único).	Sí	Formato email válido.
password	String	Contraseña encriptada.	Sí	Encriptada con bcrypt (no se expone en responses de lectura).
fecha_creacion	DateTime	Fecha de creación del usuario.	Sí (auto-generado)	Formato ISO 8601 (e.g., "2025-10-02T18:00:00Z").

Schemas de Validación (usando Joi): Se aplican en POST y PUT para asegurar integridad de datos. Errores devuelven 400 con mensajes específicos.

Endpoints

Todos los endpoints están bajo **/api/usuarios**. Se utilizan métodos HTTP estándar para operaciones RESTful.

Método	Endpoint	Descripción	Parámetros	Código de Respuesta Esperado
POST	/usuarios	Crea un nuevo usuario.	Body: JSON (nombre, email, password)	201 Created
GET	/usuarios	Lista todos los usuarios.	Ninguno	200 OK
GET	/usuarios/:id	Obtiene un usuario por ID.	:id (path)	200 OK / 404 Not Found
PUT	/usuarios/:id	Actualiza un usuario existente.	:id (path), Body: JSON (campos parciales)	200 OK / 404 Not Found
DELETE	/usuarios/:id	Elimina un usuario por ID.	:id (path)	200 OK / 404 Not Found

- **Headers comunes:** **Content-Type:** **application/json**.
- **Manejo de Errores:** Respuestas en JSON con { **"error": "Mensaje descriptivo"** } y códigos HTTP apropiados (e.g., 500 para errores internos).

Ejemplos de Uso

1. Crear Usuario (POST /api/usuarios)

Request:

```
POST http://localhost:3000/api/usuarios
Headers: Content-Type: application/json
Body:
{
  "nombre": "Juan Pérez",
  "email": "juan@example.com",
  "password": "123456"
}
```

Response (201 Created):

```
{
  "id": "2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb",
  "nombre": "Juan Pérez",
  "email": "juan@example.com",
  "password": "$2a$10$... (hash)",
  "fecha_creacion": "2025-10-02T18:00:00Z",
  "mensaje": "Usuario creado exitosamente"
}
```

Error Ejemplo (400 Bad Request - Email inválido):

```
{
  "error": "El email debe ser válido"
}
```

2. Listar Usuarios (GET /api/usuarios)

Request:

```
GET http://localhost:3000/api/usuarios
```

Response (200 OK):

```
[
  {
    "id": "2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb",
```

```
"nombre": "Juan Pérez",  
"email": "juan@example.com",  
"fecha_creacion": "2025-10-02T18:00:00Z"  
}  
]
```

(Nota: El campo *password* no se incluye por seguridad).

3. Obtener Usuario por ID (GET /api/usuarios/:id)

Request (para ID = "2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb"):

```
GET http://localhost:3000/api/usuarios/2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb
```

Response (200 OK):

```
{  
  "id": "2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb",  
  "nombre": "Juan Pérez",  
  "email": "juan@example.com",  
  "fecha_creacion": "2025-10-02T18:00:00Z"  
}
```

Error (404 Not Found):

```
{  
  "error": "Usuario no encontrado"  
}
```

4. Actualizar Usuario (PUT /api/usuarios/:id)

Request:

```
PUT http://localhost:3000/api/usuarios/2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb  
Headers: Content-Type: application/json  
Body:  
{  
  "nombre": "Juan P. Actualizado",  
  "email": "juan.perez@example.com"  
}
```

Response (200 OK):

```
{
  "id": "2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb",
  "nombre": "Juan P. Actualizado",
  "email": "juan.perez@example.com",
  "fecha_creacion": "2025-10-02T18:00:00Z",
  "mensaje": "Usuario actualizado exitosamente"
}
```

5. Eliminar Usuario (DELETE /api/usuarios/:id)

Request:

```
DELETE http://localhost:3000/api/usuarios/2f94a7c0-ff72-4a29-a7bb-4c1fd88f1dfb
```

Response (200 OK):

```
{
  "mensaje": "Usuario eliminado correctamente"
}
```

Error (404 Not Found):

```
{
  "error": "Usuario no encontrado"
}
```

Pruebas

- **Herramienta Recomendada:** Postman. Crea una colección con los endpoints anteriores, incluyendo tests para validar status codes y JSON (e.g., `pm.test("Status 200", () => { pm.response.to.have.status(200); });`).
- **Casos de Prueba:**
 - Éxitos: CRUD completo.
 - Errores: Datos inválidos (e.g., email duplicado, password corto), ID inexistente.
- **Cobertura:** Pruebas manuales cubren ~90% (exitosos y errores comunes). Para automatización, integra Jest o Mocha.

Ejecuta pruebas ejecutando el servidor y enviando requests desde Postman. Incluye capturas de pantalla de respuestas exitosas y errores en tu entrega.

Buenas Prácticas Implementadas

- **Seguridad:** Contraseñas hasheadas con bcrypt (salting automático). No exponer passwords en responses de GET.
- **Validación:** Schemas con Joi para inputs.
- **Escalabilidad:** Estructura modular; fácil agregar DB (e.g., Mongoose) o autenticación (JWT).
- **Control de Versiones:** Usa Git para commits (e.g., `git init`, `git add .`, `git commit -m "feat: initial API"`).
- **Manejo de Errores:** Middleware global para errores 500.

Lecciones Aprendidas

- La importancia de estructurar correctamente un proyecto backend.
- Cómo manejar correctamente los métodos HTTP y rutas REST.
- La utilidad de Postman para pruebas rápidas y detección de errores.
- El valor de mantener el código modular mediante controladores, modelos y rutas.